A detailed timeline of catastrophic forgetting through the weights

A circuit view in which grokking and catastrophic forgetting are two faces of the same coin

Parag Garg

Draft, October 2026

Abstract

Catastrophic forgetting — a network losing what it knew as soon as it learns something new — is among the largest obstacles to AI that keeps learning after deployment. Overcoming it needs a detailed account of what happens inside a network as it forgets. We give one, from the weights and from random-token probes — no stored data of either task — for two capabilities whose circuits are known: modular addition in a small grokked transformer, and induction (copying what followed a token's earlier occurrence) in Pythia, repeated up the Pythia family and in Qwen2.5. In both, grokking and forgetting are two faces of one coin: the circuit's connections are in place before the behaviour appears, and the behaviour is lost while most of the circuit is still there.

Building: structure leads behaviour. The toy's output-layer circuit is at 7.96× its random level while test accuracy is 14%. In Pythia's own pretraining the induction wiring — previous-token heads composing into induction heads' keys — is significant at step 512, when copying is still zero and no induction head attends to its target: in 10–13 of 16 head pairs in each of five Pythia-160m pretraining seeds, 12 at 1.4b and 14 at 2.8b, but only 4 at 410m. By step 1 000 copying is 0.44–0.69. OLMo-7B shows the same onset.

Destroying: the behaviour goes first, in two regimes. In the toy, interference moves the weights 14–88% and *rewires* the circuit: the connections between matrices weaken from the first steps of interference (10 of 10 seeds), their significant directions are counted out 10–20 steps later, and the bulk of the circuit fades about 7× more slowly than the function. In Pythia, anti-induction destroys copying (0.84 → 0.01 in 50 steps on 3 of 3 seeds) while each induction head's weights move under 0.3% (about 0.05% when copying has halved) and the wiring does not move; at such displacements Weyl's inequality lets no spectral measure move much. The loss is *distributed*. An exact path-integral account over the weights (the parts sum to the loss within 3.8% at 160m and 0.2% in four larger models) spreads it over the network, centred on the induction heads' layers; per weight, the attention keys, values and outputs carry 1.2–7× their share in all five models tested (Pythia 160m–1.4b, Qwen2.5 0.5B–1.5B). Returning the 80 top-ranked parts restores copying to 0.62–0.66 at 160m (three seeds; random parts of the same kinds and sizes: 0.17–0.30) and to 0.17–0.86 in the four larger models (random: at most 0.008). On natural text, which the update never trained on, the same damaged models still copy a repeated passage at 60–92% of their healthy gain.

At scale, forgetting needs conflict. A new task that contradicts copying (copy-with-offset) destroys it in 13 of 13 runs (to 0.000–0.002: five fine-tuning seeds at Pythia-160m, two each at 410m, 1.4b, Qwen2.5-0.5B and 1.5B); one that does not (an arithmetic rule written as text) leaves it at 0.80–0.97 in 13 of 13, including where the rule is learned to 0.96–0.99. In the toy every interfering task we tried destroyed or badly damaged addition, each contradicting it on its own inputs or competing for the same small block, so the toy cannot say whether conflict is needed; there, the loss becomes hard to undo only once the new task is learned and generalises.

The same picture up to 14B. Up the Pythia family to 2.8b and across Qwen2.5 from 0.5B to 14B, base and instruct, a conflicting update erases copying within about ten steps (fifty at 160m) while the wiring does not move and general-text perplexity barely registers the loss. A companion evaluation of 14 models in six families finds that harmless training leaves every tested skill intact where a conflict halves it, and that a wiped skill returns after a few correct steps on other items: suppressed, not deleted.

> **Code, data and the full version.** Everything behind this paper is public at github.com/paragorth/frcs-678: the full version with every table, the experiments that did not survive, and the provenance of each number (`docs/PAPER_DRAFT.md`); the measurement tools (`tools/`); and every result file (`results/live_learning/`).

1 Introduction

A model that is trained further loses skills it had (McCloskey and Cohen, 1989; French, 1999). This is why deployed models are frozen or retrained rather than taught as they go, and it is the central obstacle to continual learning. The standard remedies — replay, penalties on moving important weights, separate parameters per task — act without an account of what, in the weights, is lost, where, and when. A remedy needs to know where to act, and an alarm needs to know what to watch. This paper gives that account for two capabilities whose circuits are known, followed step by step through learning and through forgetting: modular addition in a one-layer transformer trained until it groks, and induction — copying the token that followed an earlier occurrence of the current one — in pretrained language models.

The starting point is an observation about trained weights: shuffle the entries of a trained weight matrix, and its histogram — and so its entropy — is unchanged, but the network is dead. Whatever survives the shuffle is distribution; whatever dies is organisation. Everything here is read that way, without stored data of either task: from the weights themselves, against shuffle and random-rotation nulls, and from the model's behaviour on random-token probes. Four findings follow.

Structure leads behaviour in learning. The toy's output-layer circuit is complete while the model is right 14% of the time. In Pythia's own pretraining the induction wiring is significant at step 512, when no model can copy yet — in five pretraining seeds of Pythia-160m, at 1.4b and 2.8b, and marginally at 410m; OLMo-7B shows the same onset (§4).

Forgetting has two regimes, and the weights can show only one. In the toy, interference moves the weights 14–88% and rewires the circuit: the connections between matrices weaken from the first steps while each matrix's own structure is unchanged, and the bulk of the circuit fades about 7× more slowly than the behaviour (§5). At scale, a conflicting fine-tune erases copying while each induction head moves under 0.3%. By Weyl's inequality no spectral measure can move much at such displacements, and the wiring does not move at all; the behaviour still collapses, because the output's change is a sum of many small changes aligned with the new task (§6). Building a capability is like wiring a circuit; breaking it in a large model is more like a mixing desk on which no cable is pulled but dozens of faders are each nudged down a little: the wiring is intact and the signal is gone.

At scale the loss is distributed, but it can be located. An exact path-integral account over the weights, in five models of two families, spreads the rise in copy loss thinly over the network, centred on the circuit's own layers; returning the parts it ranks first restores copying where random parts of the same kinds and sizes do not (§6.2–6.3).

At scale, forgetting needs conflict. A new task that contradicts copying destroys it in 13 of 13 runs; one that does not leaves it intact in 13 of 13, even when it is learned almost perfectly; and the damaged models still copy natural text (§6.4–6.5).

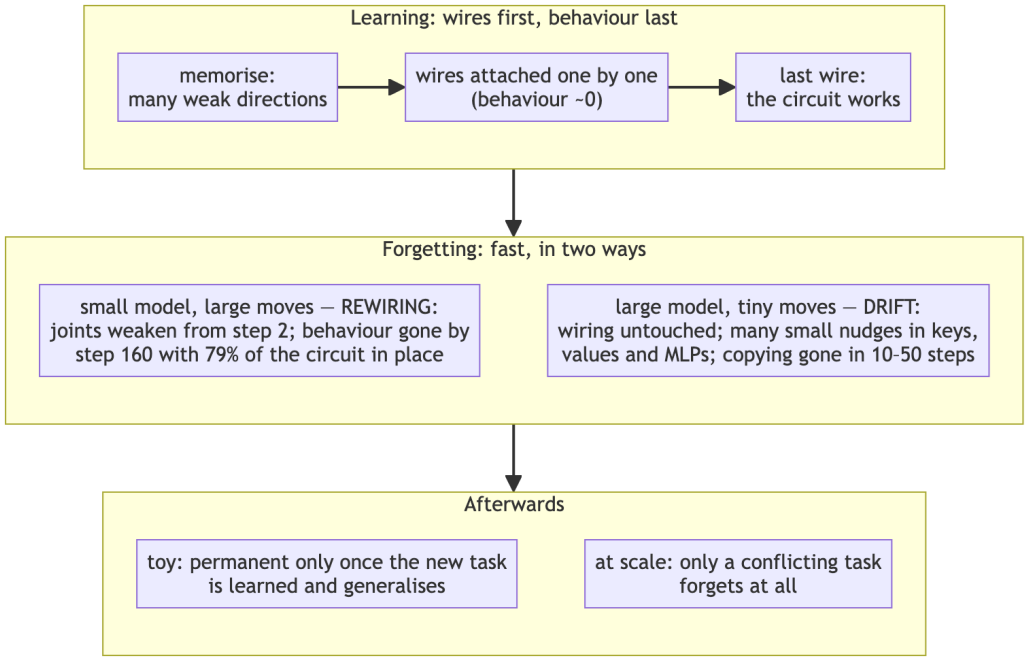


Figure 1. The timeline in one picture. Learning attaches the circuit's wires before the behaviour appears. Forgetting is fast in both regimes: a small model is rewired, with most of the circuit still in place when the behaviour is gone; in a large model the wiring is untouched and the loss is spread over many small changes. What happens afterwards depends on the new task.

2 Related work

Where forgetting happens. Ramasesh et al. (2021) locate forgetting in the higher layers of image classifiers; in induction forgetting we find it centred on the circuit's own layers, with the top layer carrying under 1%. *Forgetting as lost alignment or access* — spurious forgetting (Zheng et al., 2025), *Diagnosing Catastrophe* (Anthes et al., 2023) — is argued from activations and probes; we locate the loss in the weights and time it. *Connections between components:* composition scores (Elhage et al., 2021), low-rank channels from weight SVDs (Merullo et al., 2024), an exact Haar-rotation null scored per head (Wang, 2026) and singular-vector alignment through training (Yunis et al., 2024); ours is the dynamics, through building and forgetting. *Induction heads* form in a phase change early in training (Olsson et al., 2022); Pythia's public checkpoints (Biderman et al., 2023) and the extra pretraining seeds of PolyPythias (van der Wal et al., 2025) let us time the wiring against the behaviour. *Attribution:* integrated gradients (Sundararajan et al., 2017), attribution patching (Syed et al., 2023) and skill grafting (Panigrahi et al., 2023); we integrate over the weights, from the healthy to the damaged model, and test the ranking by restoring parts against size-matched random ones. *Suppression rather than deletion:* fine-tuned-away skills often return quickly (Jain et al., 2024; Kotha et al., 2024), which our distributed regime fits. *Same-format interference:* new facts in a familiar template damage known ones (Gekhman et al., 2024; Kaplan et al., 2026). *Perturbation bounds:* Weyl (1912); Davis and Kahan (1970). The full comparison table is in the full version.

3 Method

Structure inside a matrix. For a weight matrix W we count the directions whose singular values exceed the largest found when W's own entries are shuffled (k): structure that permutation destroys (Martin and Mahoney; Prakash and Martin, 2026). The count is stable to the null's two constants (±2 directions over their swept ranges).

Connections between matrices. A writer W (embedding, attention output, MLP output) and a reader R (queries, keys, values, MLP input, unembedding) meet in the residual stream. Their composition score $C = \|RW\|_F / (\|R\|_F \|W\|_F)$ (Elhage et al., 2021) is compared with its exact null under a Haar-random rotation of the stream, which keeps both matrices' spectra and destroys only their alignment: $E[C^2] = 1/d$, with closed-form variance (Wang, 2026). A joint's **strength** is $z = (C^2 - E[C^2]) / \mathrm{sd}$, tested with Benjamini–Hochberg

correction; its **wire count** is the number of singular values of RW above the largest under sampled rotations. At scale, **the wiring** is the K-composition from the four strongest previous-token heads into the four strongest induction heads' keys (16 pairs), with the heads identified in the final model.

What the weights can show. For any change ΔW, $|\sigma_i(W + \Delta W) - \sigma_i(W)| \le \|\Delta W\|_2 \le \|\Delta W\|_F$ (Weyl), and singular directions turn by at most about $\|\Delta W\|_2$ divided by the spectral gap (Davis–Kahan). We report the displacement $\|W_t - W_0\|_F / \|W_0\|_F$. At the 0.1–1% displacements of fine-tuning every statistic above can move only a little; the output has no such bound, since its change is a sum over every parameter of gradient $\times$ change.

Probes. 64 random-token sequences of 40 tokens, each shown twice; **copying** is next-token accuracy on the second copy; head scores are attention to the induction target and to the previous token, and the attention to the first position (the sink). A natural-text check reads Olsson et al.'s in-context learning score (loss on tokens 400–510 minus tokens 40–63 of 512-token wikitext windows) and copying of repeated wikitext passages.

Exact attribution and restore. Along the straight path from the healthy weights θ_0 to the damaged θ_1, $L(\theta_1) - L(\theta_0) = \int_0^1 \nabla L(\theta(\alpha)) \cdot (\theta_1 - \theta_0)\, d\alpha$ exactly. Splitting the dot product by parts — each head's queries, keys, values and output columns; each layer's MLP input and output; norms; embeddings — gives each part's share (midpoint quadrature, $K = 16$; the residual is reported). The causal check returns the N top-ranked parts to their healthy values in the damaged model and compares with random parts of the same kinds and sizes.

Models and training. *Toy:* a one-layer transformer ($d = 128$, four heads) on arithmetic mod 113, full-batch AdamW (learning rate 1e-3, weight decay 1.0); addition on 30% of pairs is grokked, then an interfering task on the same inputs — multiplication (25% of pairs) unless stated — is trained alone. *At scale:* Pythia 160m–2.8b and Qwen2.5 0.5B–14B (instruct and base), full fine-tuning in float32 at 3e-6 with batch 8 — never LoRA, which would constrain the very update being measured. Interfering tasks: **anti-induction** (each copied target replaced by a fixed permutation of it), **copy-with-offset** (predict the token two after the earlier occurrence), an **arithmetic rule** written as text (no conflict with copying), and wikitext as a harmless control. *Pretraining checkpoints:* Pythia's public checkpoints at 18 steps for four sizes, four extra pretraining seeds of Pythia-160m (PolyPythias), and the nine snapshots of OLMo-7B-0724.[1]

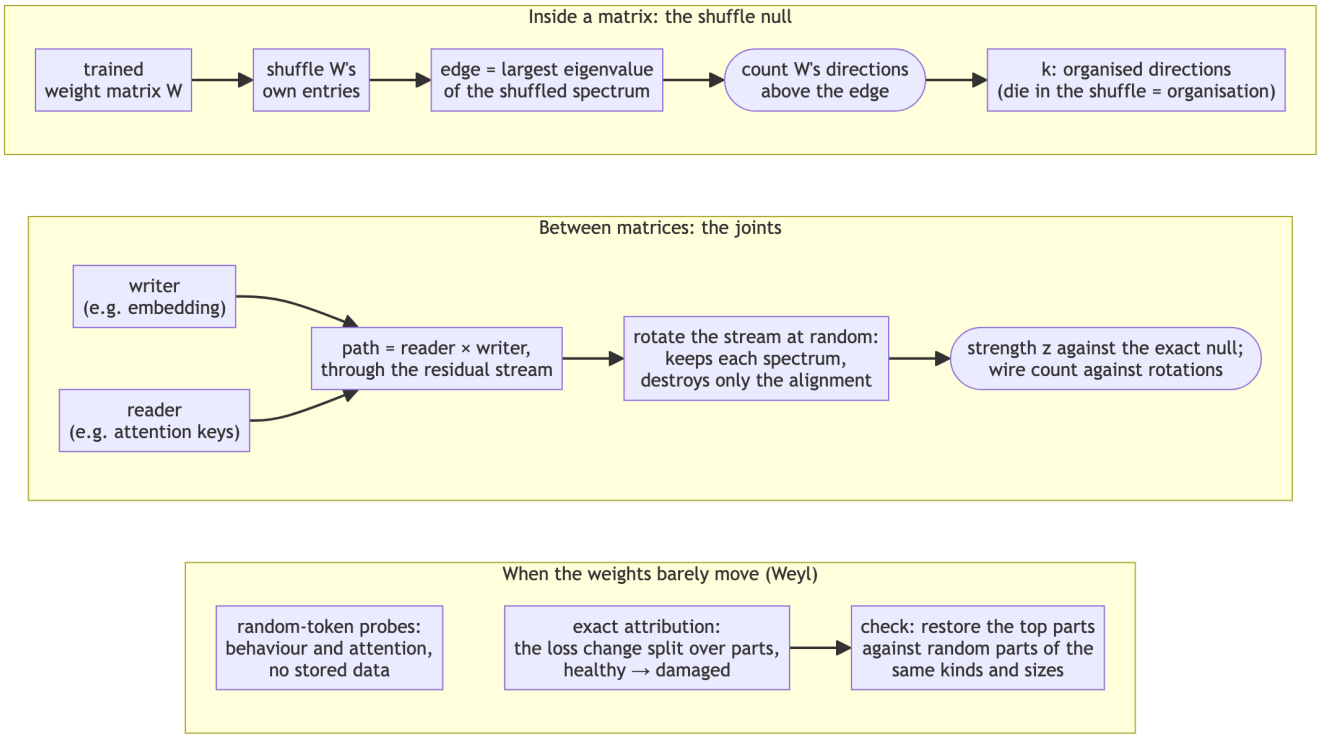


Figure 2. The measures. Inside a matrix, the shuffle null counts organised directions; between matrices, a random rotation of the shared stream calibrates each connection; when the weights barely move, random-token probes and an exact attribution read the loss instead.

[1] On the Hugging Face hub, Pythia-2.8b's revisions for steps 0–80 000 list the final model's single-file weights; only their sharded files (and the `.bin` of 40 000 and 80 000) differ per step, so a default load of an early revision returns the finished model. We use the file unique to each revision.

4 Learning: the wiring comes first

In the toy, trained on addition from scratch, the training set is memorised by epoch 1 000 and test accuracy stays near zero until about epoch 35 000:

epoch	**20 000**	**30 000**	**35 000**	**40 000**
test accuracy	0.009	0.034	**0.140**	1.000
output-layer Fourier concentration (× random)	2.56	6.63	**7.96**	6.10

The output layer's circuit is complete while the model is right 14% of the time; in matrix terms the transition goes from many weak directions to a few strong ones (Figure 4, panels a–b).

At scale, in each model's own pretraining:

run	**step 512: copying**	**step 512: wiring pairs significant**	**step 1 000: copying**
Pythia-160m, five pretraining seeds	0.000	10–13 of 16	0.59–0.61
Pythia-410m	0.000	4 of 16	0.44
Pythia-1.4b	0.000	12 of 16	0.69
Pythia-2.8b	0.000	14 of 16	0.60

In every Pythia run the wiring is significant at step 512 while copying is exactly zero and no induction head yet attends to its target (best score 0.02–0.05, against 0.85–0.96 at step 1 000). The lead is bounded by the checkpoint spacing — none exist between steps 512 and 1 000 — and at 410m it is marginal. In OLMo-7B, at step 1 000 (4B tokens) 3 of 16 pairs are significant while copying is 0.010; by step 5 000 copying is 0.78 with 7 of 16, and the wiring on the final model's heads completes only by step 50 000: the onset leads, the full circuit does not. The weights then settle: movement per step falls 37–71-fold from the phase change to the end of training.

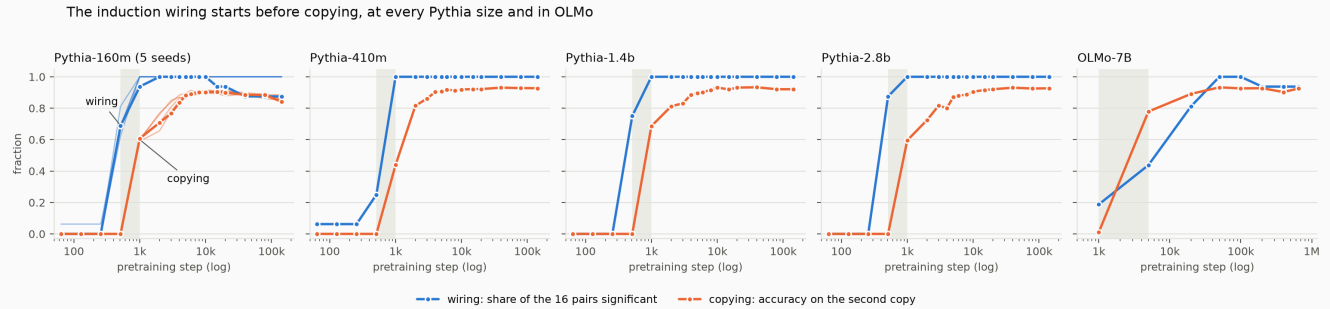


Figure 3. The induction wiring starts before copying at every Pythia size and in OLMo. Copying (orange) and the share of the 16 previous-token → induction-key pairs that are significant (blue), through each model's own pretraining; the five Pythia-160m pretraining seeds are overlaid, the original in bold. Grey: the interval in which copying appears and no checkpoint exists.

5 Forgetting in a small model: rewiring

Every interfering task we tried destroyed or badly damaged addition, and every one was posed on addition's own inputs or competed for the same small block. Under multiplication (10 seeds; a control keeps training addition, 3 seeds):

step	0	2	3	10	20	160
addition	1.00	0.99	0.84	0.87	0.54	0.05
embedding → keys strength z	27.2	25.2	24.3	18.5	16.3	22.6
its wire count	3	3	3	3	1.5	1
control strength z	27.2	27.0	26.9	26.2	26.0	29.3

The connection's strength is more than 5% below its start at step 2 in 10 of 10 seeds, one step before accuracy first falls below 0.90; the control stays within 5% until step 10. The discrete wire count lags 10–20 steps. Each matrix keeps its own count of organised directions while its contents are rewritten (the keys move 57% by step 20). The circuit fades about 7× more slowly than the behaviour: 79% of the output-layer circuit is still present at step 160, with addition at 0.05, and it takes about 1 000 more steps to fall to the level of memorisation.

Whether the loss lasts is a separate, later event. In 31 of 32 runs, four events come in order: addition is destroyed (by ∼200 steps) while each matrix's organised directions are unchanged; multiplication is learned (∼1 900–4 600); the old directions collapse (∼4 400–5 600); and only then does the loss become hard to undo. In all 8 runs where multiplication failed to generalise, addition came back. Repair needs a set of parts: restoring attention and MLP together recovers in 15 of 15 runs, single parts erratically, and the two ends of the first connections to weaken — the keys and the MLP output — no better than restoring nothing.

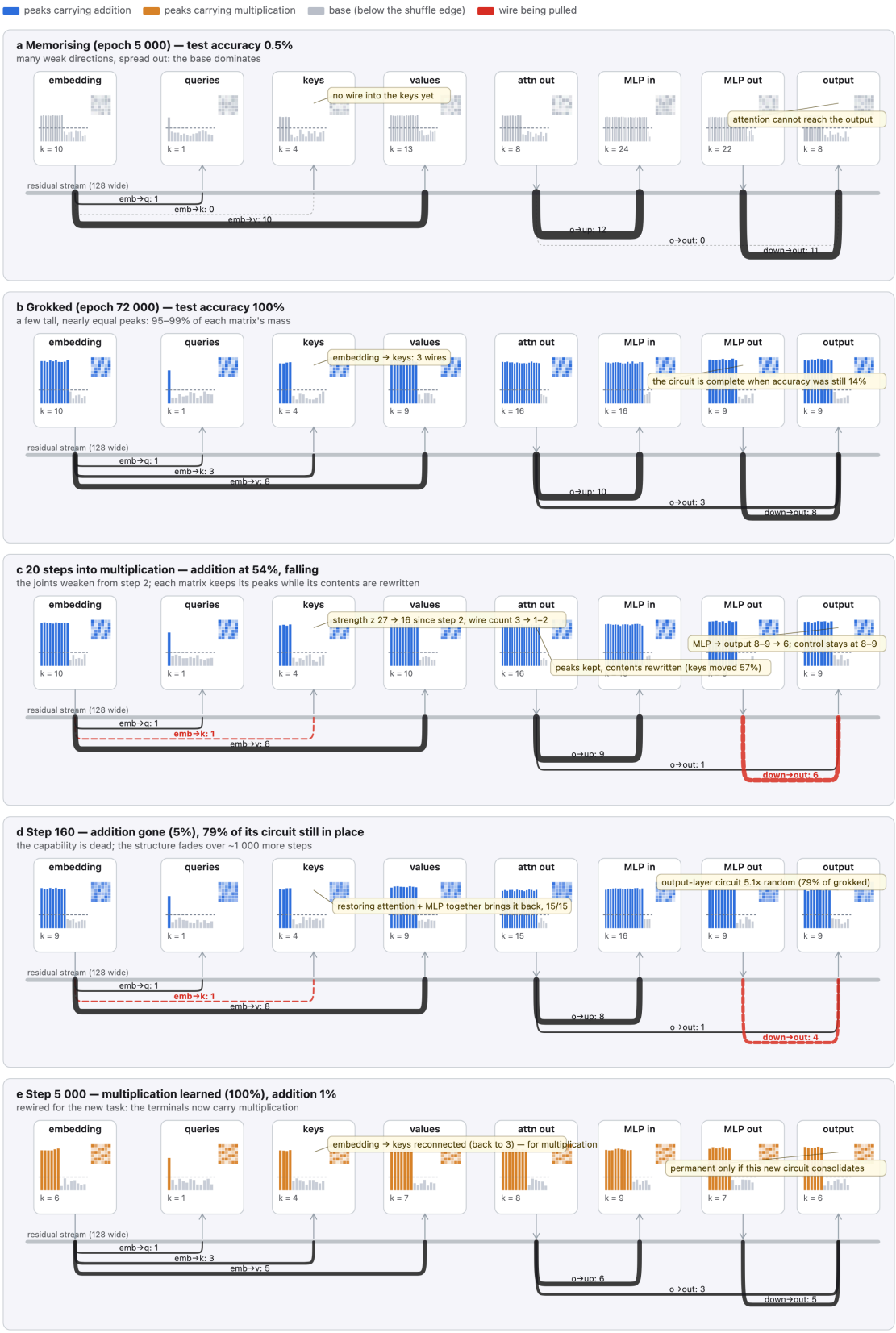


Figure 4. One circuit, built wire by wire and broken fast (toy, addition then multiplication; seed means). Each node is a weight matrix — its spectrum, with peaks above the shuffle edge (organised directions) and the base below — and wires run through the shared residual stream, their thickness the wire count. (a) Memorising: many weak directions. (b) Grokked: a few tall, nearly equal peaks. (c) 20 steps into multiplication: the joints have weakened since step 2 while each matrix keeps its peaks. (d) Step 160: addition gone, 79% of its circuit still in place. (e) Step 5 000: rewired, the same terminals now carrying multiplication.

6 Forgetting at scale: distributed drift

6.1 Copying goes, the wiring does not move

Anti-induction at 3e-6 takes Pythia-160m's copying from 0.84 to 0.01 in 50 steps (three fine-tuning seeds: 0.28 / 0.25 / 0.26 at step 10, 0.010 / 0.009 / 0.006 at step 50) with the wiring at z = 115.9 at every mark. The same happens at every size, in both families:

model	copying, step 0	step 5	step 10	wiring z (unchanged)	perplexity, step 0 → 10
Pythia-160m	0.841	0.60	0.25–0.28	115.9	88 → 91
Pythia-410m	0.927	0.306	0.025	186.4	74.0 → 80.3
Pythia-1.4b	0.921	0.095	0.009	349.2	60.4 → 60.7
Pythia-2.8b	0.927	0.069	0.008	364.0	52.1 → 50.9
Qwen2.5-0.5B-Instruct	0.936	0.531	0.023	73.7	84.1 → 81.1
Qwen2.5-1.5B-Instruct	0.948	0.767	0.034	63.2	101.6 → 95.3
Qwen2.5-3B-Instruct	0.943	0.625	0.029	238.3	105.3 → 98.7
Qwen2.5-7B-Instruct	0.948	0.528	0.005	372.5	58.7 → 53.4
Qwen2.5-7B (base)	0.949	0.551	0.002	372.7	18.4 → 19.3
Qwen2.5-14B-Instruct	0.938	0.050	0.000	613.7	55.2 → 52.4
Qwen2.5-14B (base)	0.949	0.039	0.000	613.7	16.1 → 13.1

Each induction head's keys move about 0.05% when copying has halved and under 0.3% by step 50; a wikitext control at the same rate leaves copying intact. General-text perplexity barely registers the loss — in the Qwen instruct models it improves while copying dies. The induction heads' attention slides to the attention sink, but restoring their queries and keys returns copying only to 0.014: a symptom, not the seat. And each model breaks at its own displacement of those keys, which the learning rate barely changes (the full version gives it at every size).

6.2 An exact account of the loss

The path integral attributes the rise in copy loss (50 damaging steps) to the parts, in five models. Per-weight blame is a kind's share of the rise divided by its share of the attention and MLP weights:

model	residual	queries	keys	values	attention out	MLP in	MLP out	largest part
Pythia-160m (3 seeds)	+1.6 to +3.8%	0.37	**4.1**	2.1–2.2	1.2–1.3	0.49	0.65–0.70	5.8–6.8%
Pythia-410m	+0.1%	0.67	**2.6**	1.8	1.3	0.49	0.91	2.9%
Pythia-1.4b	+0.2%	0.80	1.3	**2.1**	1.8	0.48	1.05	4.0%
Qwen2.5-0.5B	0.0%	1.2	3.7	**7.1**	3.0	0.71	0.97	3.7%
Qwen2.5-1.5B	0.0%	1.5	4.3	**6.3**	3.4	0.69	0.90	3.3%

The account closes. No single part carries more than 7%; the loss centres on the induction heads' layers and is near zero in the first and last. Per weight, keys, values and attention outputs carry more than their size in every model, while the MLP inputs carry about half theirs.

6.3 The account is causal

Parts returned to their healthy values in the damaged model (share of the attention and MLP weights in brackets):

	Pythia-160m (3 seeds)	**410m**	**1.4b**	**Qwen 0.5B**	**Qwen 1.5B**
healthy → damaged	0.84 → 0.01	0.93 → 0.00	0.92 → 0.00	0.94 → 0.00	0.95 → 0.00
best single kind, in full	≤ 0.07	0.00	0.00	0.00	0.00
top 80 parts by attribution	**0.62–0.66** (62–64%)	**0.37** (60%)	**0.17** (52%)	**0.78** (61%)	**0.86** (57%)
80 random parts, same kinds and sizes	0.17–0.30	≤ 0.006	0.002	≤ 0.008	0.002

The comparison has to match size: at 160m the top 80 parts are 8% of the parts but 64% of the weights, and random parts of any size restore nothing simply because they are small. Against size-matched random parts the ranking wins 2–6× at 160m and restores 0.17–0.86 against at most 0.008 in the larger models. No single kind repairs the loss; a set does, as in the toy.

6.4 Forgetting needs conflict

model (fine-tuning seeds)	**copy-with-offset: copying after**	**arithmetic rule: rule learned → copying after**
Pythia-160m (5)	0.000–0.002	0.47–0.55 → 0.80–0.88
Pythia-410m (2)	0.001	0.58–0.76 → 0.92–0.93
Pythia-1.4b (2)	0.000	0.96–0.99 → 0.93–0.94
Qwen2.5-0.5B (2)	0.000–0.001	0.87 → 0.96
Qwen2.5-1.5B (2)	0.000	0.96–0.99 → 0.97

Copy-with-offset, learned to 0.93–1.00 everywhere, takes copying to zero in 13 of 13 runs; the rule leaves it intact in 13 of 13, including where it is learned almost perfectly. Under offset the same parts carry the old skill's loss and the new task's gain (at 160m the same three keys in all five seeds). The harmless wikitext control erodes copying only slowly (0.84 → 0.74 after 150 steps). "Conflict" is defined by construction here, and the toy cannot test whether it is needed, since every task we tried there destroyed addition. In a companion stream of synthetic skills, conflict also arrives by generalisation: a new skill sharing an old one's answer format captures its prompts.

6.5 On natural text the loss is partial

The damaged models of §6.2, read on wikitext, which the damaging update never saw:

model	**random-token copying, damaged**	**natural-text copying, second copy: healthy → damaged**	**gain kept**	**in-context learning score: healthy → damaged**
Pythia-160m, 3 seeds	0.006–0.009	0.930 → 0.646–0.682	63–68%	−0.97 → −1.06 to −1.14
Pythia-410m	0.000	0.941 → 0.668	64%	−0.93 → −0.53
Pythia-1.4b	0.000	0.939 → 0.657	60%	−0.97 → −1.29
Qwen2.5-0.5B	0.000	0.950 → 0.804	79%	−0.72 → −0.78
Qwen2.5-1.5B	0.000	0.955 → 0.897	92%	−0.79 → −0.78

Copying a repeated passage keeps 60–92% of its gain over the first copy, and the in-context learning score is unchanged or stronger in six of seven (the exception, Pythia-410m, is also where 50 steps nearly triple general-text perplexity). Copying is gone on the kind of input the update trained on and only partly lost

elsewhere — the pattern of suppression rather than deletion.

7 Discussion: one circuit picture, two regimes

What is common. In both regimes structure leads behaviour in learning; the behaviour dies while most of the circuit is still in place; the attention side that decides where heads look and what they carry takes more of the loss than its size; and no single part repairs the loss, while a set does. Grokking and forgetting are two faces of the same coin because building needs every connection, while destroying needs only that the circuit's output be redirected.

What differs is how far the weights must move. A skill halves when about 60% of the toy's keys have moved, and when about 0.06% of Pythia's induction keys have — a thousandfold apart. This is mostly a property of the model, not of the learning rate: over a 100-fold range of rates the toy forgets at the same displacement, and Pythia-160m at rates 10–33× higher is destroyed within two steps with its wiring unchanged. Weyl then decides what the weights can show: at the toy's displacement every spectral measure can move; at Pythia's none can move much.

What it means for intervening. In the rewiring regime a weights-only alarm is possible, because the connections move with the behaviour. At scale an alarm has to read behaviour, and random-token probes do so without stored data. Repair has to act on a set of parts, and the account says which: the parts it ranks first, at the circuit's own layers.

8 Limitations

One toy architecture (a one-layer transformer) and one circuit at scale, measured on random-token copying. The attribution is computed at one dose (50 steps at 3e-6), on three fine-tuning seeds at 160m and one elsewhere. The conflict result rests on two to five fine-tuning seeds per cell, and "conflict" is defined by construction. The pretraining lead is bounded by the checkpoint spacing, and at 410m it is marginal. The comparison of the two regimes rules out the learning rate as their cause, but not size or the other differences between a one-layer toy and a pretrained family.

9 Conclusion

A capability in a transformer is built ahead of its behaviour and lost fast, and the weights record both — in two different ways. In learning, the circuit's wiring is in place before the behaviour appears. In forgetting, the behaviour goes first: a small network is rewired, and the connections between its matrices show the break as it happens; in a large one the update is a thousandth the size, no spectral measure can move much, and the loss is spread thinly over many parts — which an exact account over the weights locates and a size-matched restore confirms. At scale, fast forgetting needed conflict. All of it is read from the weights and from random-token probes, with no stored data.

Full version, tools and results: github.com/paragorth/frcs-678. References are being compiled with the citation check.